SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and	5

	justify your decisions. Explain how your approach enhances reliability and reduces risk.	
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5
4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5

5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that	5

	maximizes defect detection while minimizing redundant test cases.	
9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5

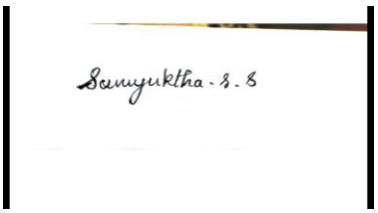
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5
15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I Samyuktha S S(192311056) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

ANSWERS

1. Online Banking System – Boundary Value Analysis and Equivalence Partitioning

Problem Analysis:

An online banking system experiences frequent failures during fund transfers, especially at minimum and maximum transfer limits. This indicates that the existing test cases do not adequately cover boundary values. Random testing may detect some errors, but it cannot guarantee that all important input conditions are tested. Therefore, a structured black-box testing approach is required.

1. Equivalence Class Partitioning (ECP)

Equivalence Class Partitioning divides input data into groups that are expected to behave similarly. Instead of testing every possible amount, one representative value from each group is selected.

Assume the permitted transfer amount is ₹100 to ₹50,000.

- Invalid class 1: Amounts below ₹100, such as ₹50.
- Valid class: Amounts from ₹100 to ₹50,000, such as ₹5,000.
- Invalid class 2: Amounts above ₹50,000, such as ₹60,000.

The system should reject invalid amounts and process valid amounts according to the banking rules.

2. Boundary Value Analysis (BVA)

Boundary Value Analysis focuses on values at and around the limits because errors frequently occur at these points.

Test case	Transfer amount	Expected result
1	₹99	Reject
2	₹100	Accept
3	₹101	Accept
4	₹49,999	Accept
5	₹50,000	Accept
6	₹50,001	Reject

These test cases help identify errors such as incorrect comparison operators and improper limit validation.

3. Evaluation of Random Testing

Random testing selects inputs without following a fixed pattern. It can discover unexpected defects, but it may fail to test exact boundary values or important combinations consistently. Therefore, random testing should be retained as a supplementary technique rather than used as the primary strategy.

4. Improved Testing Strategy

The testing team should combine ECP and BVA with tests for insufficient account balance, invalid account numbers, network failures, and repeated transfer requests. Automated test cases should verify the transaction status, account balance, and transaction history. Regression testing should be performed after every correction.

Conclusion:

Combining Equivalence Class Partitioning, Boundary Value Analysis, and supplementary random testing provides systematic input coverage. This approach improves defect detection, prevents incorrect fund transfers, and increases the reliability and security of the online banking system.

2. Healthcare Application – White-Box Testing and Coverage

Problem Analysis:

A healthcare application has experienced five critical defects after deployment due to insufficient test coverage. This suggests that the testing team may have focused on executing individual statements without adequately testing decision outcomes and execution paths. Since healthcare applications handle important patient information and treatment decisions, thorough white-box testing is essential.

1. Control Flow Graph (CFG)

A Control Flow Graph represents the flow of execution through a program. Nodes represent statements or processing steps, while edges represent the possible transitions between them.

A conceptual patient-processing module can be represented as follows:

Start → Receive patient details → Validate patient information.

- If the information is invalid, display an error and stop processing.
- If the information is valid, check whether the patient requires emergency treatment.
- If the patient is an emergency case, assign high priority.
- Otherwise, assign normal priority.
- Save the patient record and display the result.

The CFG should represent both outcomes of each decision. Testing only the valid-input and emergency paths would leave other paths untested.

2. Analysis of Coverage Criteria

a) Statement Coverage: Ensures that every executable statement is executed at least once. However, executing all statements does not guarantee that every decision outcome has been tested.

b) Branch Coverage: Ensures that every decision branch, including true and false outcomes, is executed. It is more effective than statement coverage for identifying missing decision tests.

c) Path Coverage: Examines different execution paths through the program. For example, invalid patient information, valid emergency cases, and valid non-emergency cases should follow different paths. Complete path coverage may be impractical when loops create many possible paths.

3. Missing Test Cases

The testing team should examine cases such as:

- Missing or invalid patient information.
- Valid patient information with emergency status.
- Valid patient information without emergency status.
- Invalid data combined with emergency status.
- Database failures while saving patient records.

These cases help reveal defects in validation, prioritization, and record processing.

4. Improved Testing Strategy

First, construct the CFG and identify all decision points. Next, design test cases to cover every branch and important independent path. Measure statement and branch coverage using suitable testing tools. Prioritize emergency-related paths and perform regression testing after fixing defects.

Conclusion:

A structured white-box testing strategy improves control-flow coverage and identifies hidden logical errors. By combining branch testing, path testing, and risk-based prioritization, the healthcare application can become more reliable and reduce the possibility of critical post-deployment failures.

3. Retail E-Commerce System – Black-Box Testing

Problem Analysis:

A retail e-commerce system accepts inputs such as price, quantity, and discount. The testing team failed to identify errors caused by particular combinations of these inputs. This indicates that testing individual fields alone is insufficient. The system must be tested for both individual input validity and interactions between inputs.

1. Equivalence Class Partitioning (ECP)

ECP divides inputs into valid and invalid categories so that representative values can be tested.

For example:

- Price: Positive values are valid; zero and negative values are invalid if the business rules require a positive price.
- Quantity: Positive whole numbers are valid; zero, negative numbers, and unsupported fractional quantities are invalid.
- Discount: Values within the permitted discount range are valid; negative discounts and discounts exceeding the maximum are invalid.

Representative values from each class should be selected to reduce unnecessary test cases while maintaining useful coverage.

2. Boundary Value Analysis (BVA)

BVA tests values at and around input limits.

For example, if the permitted discount range is 0% to 50%, the following values should be tested: −1%, 0%, 1%, 49%, 50%, and 51%.

Similarly, test the minimum permitted price and quantity, as well as values just below and above their limits. These tests help detect errors in validation and calculations.

3. Decision Table Testing

Decision Table Testing is useful when the output depends on combinations of multiple conditions.

Price	Quantity	Discount	Expected result
Valid	Valid	Valid	Calculate total correctly
Invalid	Valid	Valid	Reject invalid price
Valid	Invalid	Valid	Reject invalid quantity
Valid	Valid	Excessive	Reject invalid discount

Valid	Valid	Maximum permitted	Apply the permitted discount

The system should also be tested with combinations involving multiple invalid inputs and special discount rules.

4. Improved Testing Strategy

The testing team should combine ECP, BVA, and Decision Table Testing. It should verify the final amount using the formula:

Total Amount = (Price × Quantity) − Discount Amount.

Tests should cover normal purchases, maximum discounts, invalid quantities, and interactions between pricing rules. Pairwise testing can also help cover interactions between several input parameters without testing every possible combination.

Conclusion:

Combining these black-box techniques improves input and rule coverage. It helps identify calculation errors, invalid combinations, and discount-related defects while reducing redundant test cases. This results in a more accurate and reliable e-commerce system.

4. Software Module – Path Testing and Control Flow Graph

Problem Analysis:

A software module contains complex nested conditions and loops, but the testing team reports only partial path coverage. This indicates that some possible execution routes have not been adequately tested. Missing paths may contain logical errors, incorrect loop conditions, or unexpected behavior.

1. Control Flow Graph Construction

A Control Flow Graph (CFG) represents the execution structure of a program. It contains nodes for statements and decisions, and directed edges for possible transitions.

For a module containing nested conditions and loops, the CFG should include:

- Entry and exit nodes.
- Conditional statements with true and false branches.
- Nested decision points.
- Loop entry, iteration, and exit edges.
- Connections between successive processing steps.

An incomplete CFG can cause the team to overlook important branches and independent paths.

2. Independent Path Identification

An independent path is an execution path that introduces at least one new edge not included in previously identified paths. Cyclomatic complexity helps estimate the number of independent paths required for basis path testing.

The formula is:

$$V(G) = E - N + 2P$$

Where:

- (V(G)) = Cyclomatic complexity.
- (E) = Number of edges.
- (N) = Number of nodes.
- (P) = Number of connected components, usually 1 for a single connected program.

For example, if a CFG has 8 edges, 7 nodes, and 1 connected component, then:

$$V(G) = 8 - 7 + 2(1) = 3$$

This indicates three independent basis paths.

3. Testing Nested Conditions and Loops

The testing team should create test cases for both outcomes of every decision. Loops should be tested with zero iterations, one iteration, and multiple iterations wherever applicable. Nested conditions should be tested with different combinations of true and false outcomes.

The team should also examine loop termination, invalid inputs, and exception-handling paths.

4. Improved Testing Strategy

Construct and verify the CFG, calculate cyclomatic complexity, identify independent paths, and design test cases for each path. Measure statement and branch coverage, and prioritize paths associated with critical functions. Complete path coverage should be attempted where feasible, but complex loops may create too many possible paths.

Conclusion:

Systematic path testing improves the detection of logical errors in complex control structures. CFG analysis and cyclomatic complexity provide a structured way to identify important paths, improve coverage, and increase software reliability.

5. Financial Application – Hybrid Testing Strategy

Problem Analysis:

A financial application uses both black-box and white-box testing, yet defects continue to appear in production. This suggests that the existing test cases may not adequately cover input boundaries, internal decision logic, and important execution paths. A better-integrated testing strategy is required to improve defect detection and transaction reliability.

1. Black-Box Testing Techniques

Black-box testing evaluates system behavior without examining the internal source code.

a) Equivalence Class Partitioning (ECP): Divide transaction inputs into valid and invalid groups. For example, transfer amounts within the permitted limit form a valid class, while negative amounts and amounts exceeding the maximum limit form invalid classes.

b) Boundary Value Analysis (BVA): Test values immediately below, at, and above the minimum and maximum transfer limits. Also test account balances around the amount required for a transaction.

These techniques help identify invalid input handling and errors at financial limits.

2. White-Box Testing Techniques

White-box testing examines the internal structure and execution logic of the application.

a) Control Flow Graph (CFG): Construct a graph representing transaction validation, balance checking, authorization, fund transfer, and error handling.

b) Path Testing: Test important independent paths, such as successful transfers, insufficient-fund failures, invalid account details, and transaction failures.

c) Coverage Analysis: Measure statement and branch coverage to identify untested statements and decision outcomes. Prioritize critical transaction paths and exception-handling logic.

3. Evaluation of Test Adequacy

Using both testing methods does not automatically guarantee adequate testing. The team must ensure that test cases are based on requirements, boundaries, decision outcomes, and risks. High statement coverage alone is insufficient because some branches and input combinations may remain untested.

4. Improved Hybrid Testing Strategy

The team should follow these steps:

1. Identify transaction requirements and define valid and invalid input classes.
2. Apply ECP and BVA to transaction amounts, balances, and account details.
3. Construct a CFG and identify independent paths.
4. Test successful transfers, insufficient balances, rejected transactions, and network failures.
5. Automate regression tests and verify transaction records and account balances.
6. Review coverage reports and add test cases for uncovered critical paths.

Conclusion:

A properly integrated hybrid testing strategy evaluates both external behavior and internal program logic. Combining ECP, BVA, CFG analysis, path testing, and regression testing improves coverage, detects defects earlier, and reduces the risk of financial errors in production.

6. HR Payroll System – Boundary Value Analysis and Equivalence Class Partitioning

Problem Analysis:

A specialized HR payroll system calculates monthly bonuses based on employees' years of service and performance ratings. Employees with exactly 5, 10, and 15 years of service receive incorrect payouts. This indicates possible errors in the conditions used to calculate bonuses, especially at threshold values. Therefore, systematic black-box testing techniques are required.

1. Boundary Value Analysis (BVA)

Boundary Value Analysis tests values immediately below, at, and above important limits. The following test cases can identify errors in bonus calculations.

Service threshold	Test values (years)
5 years	4, 5, 6
10 years	9, 10, 11
15 years	14, 15, 16

The expected bonus for each value must be verified against the organization's bonus policy. Particular attention should be given to exactly 5, 10, and 15 years, where the reported errors occur.

2. Equivalence Class Partitioning (ECP)

ECP divides inputs into groups that are expected to produce similar results.

- Valid service duration: Employees whose service falls within the permitted range.
- Invalid service duration: Negative values or values outside the permitted range.
- Valid performance ratings: Ratings accepted by the organization's policy.
- Invalid performance ratings: Missing, unsupported, or out-of-range ratings.

Representative test cases should be selected from each class.

3. Improved Testing Strategy

Test every service threshold with different performance ratings. Verify the bonus amount, eligibility conditions, and final payroll calculation. Include invalid employee records and regression testing after correcting the defects.

4. Justification

Exploratory testing relies on the tester's experience and may overlook exact threshold values. BVA systematically checks boundary conditions, while ECP reduces redundant tests by grouping similar inputs.

Conclusion:

Combining BVA and ECP improves the accuracy of payroll testing. These techniques help detect boundary-related calculation errors and ensure that employees receive bonuses according to the organization's rules.

7. Laboratory Temperature Control – Control Flow Graph and MC/DC

Problem Analysis:

A laboratory temperature control system uses complex if-else conditions to control heaters, coolers, and alarms. After deployment, the alarm failed to activate during a sensor malfunction. This indicates that important decision combinations or branches may not have been tested adequately. Since the system operates in a safety-critical environment, thorough white-box testing is essential.

1. Control Flow Graph (CFG)

A conceptual control flow can be represented as follows:

Start → Read temperature and sensor status → Check sensor validity.

- If the sensor is faulty, activate the alarm and enter a safe state.
- If the sensor is valid, compare the temperature with the permitted range.
- If the temperature is too low, activate the heater.
- If the temperature is too high, activate the cooler.
- If the temperature is within the permitted range, maintain the normal state.
- End the control cycle and repeat monitoring.

The CFG must include all decision outcomes, including sensor failure and normal operation.

2. Branch Coverage

Branch coverage ensures that every decision outcome is executed at least once. The team should test valid sensors, faulty sensors, high temperatures, low temperatures, and normal temperatures. However, branch coverage alone may not reveal every defect involving combinations of Boolean conditions.

3. Modified Condition/Decision Coverage (MC/DC)

MC/DC requires each individual Boolean condition to independently affect the overall decision outcome.

Consider the alarm decision:

Alarm = SensorFault OR OverTemperature

Sensor fault	Over-temperature	Alarm

False	False	False
True	False	True
False	True	True

These cases demonstrate that changing either condition independently can change the alarm decision.

4. Improved Testing Strategy

Construct the CFG, identify all decision conditions, and design test cases to achieve 100% MC/DC for the defined logic. Test sensor disconnections, abnormal readings, simultaneous faults, and recovery after faults. Verify that the alarm activates correctly and that the system enters a safe state.

Conclusion:

Combining CFG analysis, branch coverage, and MC/DC provides strong logical coverage. It helps detect missed alarm conditions, improves fault handling, and increases the reliability of the laboratory temperature control system.

8. Travel Booking Portal – Decision Table Testing

Problem Analysis:

A travel booking portal allows users to apply multiple promotional codes, but certain combinations produce a negative total amount. This indicates that the current testing approach does not adequately examine interactions between ticket price, promotional rules, and user membership tiers. Testing each input independently is insufficient because the defect may occur only when several conditions interact.

1. Decision Table Testing

Decision Table Testing organizes combinations of conditions and their expected actions. The important conditions include ticket price validity, promo code validity, user eligibility, and whether multiple codes can be combined.

Test condition	**Expected result**
Valid price and eligible promo	Apply the permitted discount
Invalid or expired promo	Reject the promo code
Eligible user tier	Apply the applicable tier benefit
Incompatible promo codes	Reject the invalid combination
Discount exceeds permitted amount	Reject or cap the discount according to policy
Calculated total becomes negative	Prevent the negative total

2. Cause-Effect Graphing

Cause-Effect Graphing identifies relationships between input conditions and system outputs. For example, a valid promotional code and an eligible user tier may cause a discount to be applied.

However, an expired code should not reduce the price. The graph can then be converted into test cases.

3. Improved Testing Strategy

First, define all promotional rules and identify valid and invalid input combinations. Next, design tests for individual codes, compatible combinations, incompatible combinations, and different user tiers. Verify that discounts are calculated correctly and that the final amount never falls below the permitted minimum.

Use pairwise testing where appropriate to reduce redundant combinations, while retaining tests for critical business rules.

4. Justification

Testing inputs in isolation cannot reveal every interaction defect. Decision tables and cause-effect graphs explicitly represent relationships between conditions, helping testers identify missing combinations.

Conclusion:

Decision Table Testing and Cause-Effect Graphing provide systematic coverage of promotional rules. They help detect discount conflicts, prevent negative totals, reduce redundant test cases, and improve the reliability of the travel booking portal.

9. Database Migration Tool – Path Testing and Cyclomatic Complexity

Problem Analysis:

A legacy database migration tool uses deeply nested loops to validate records. Although the team reports high statement coverage, the application frequently crashes because of infinite loops. This demonstrates that executing every statement does not guarantee that loops terminate correctly or that all control-flow paths have been tested.

1. Limitations of Statement Coverage

Statement coverage confirms that every executable statement runs at least once. However, a loop may execute successfully for one input and fail to terminate for another. Therefore, high statement coverage can coexist with serious loop-control defects.

2. Control Flow Graph and Path Testing

The team should construct a Control Flow Graph (CFG) showing record validation, nested loops, conditional checks, and loop exit conditions.

Basis Path Testing should then identify independent execution paths. Test cases should cover valid records, invalid records, empty databases, and different record counts.

3. Loop Boundary Testing

The following conditions should be tested:

- Zero iterations, where the loop should be skipped.
- One iteration, to verify basic processing.
- Multiple iterations, to verify repeated processing.
- Empty or malformed records, to test error handling.
- Large datasets, to examine termination and performance.

The team should also verify that loop variables are updated correctly and that every loop has a reachable exit condition.

4. Cyclomatic Complexity

Cyclomatic complexity estimates the number of independent paths in a program. It can be calculated using:

$$V(G) = E - N + 2P$$

Here, (E) is the number of edges, (N) is the number of nodes, and (P) is the number of connected components.

A higher value indicates greater control-flow complexity and generally increases the testing effort required. The metric helps identify independent paths but does not, by itself, guarantee that all loops terminate.

5. Improved Testing Strategy

Use CFG analysis, basis path testing, loop-boundary testing, and automated timeout checks. Verify that every record is processed exactly as intended and that migration results match the source data. Perform regression testing after fixing loop defects.

Conclusion:

Statement coverage alone is insufficient for nested-loop programs. Combining path testing, loop-boundary analysis, and cyclomatic complexity helps identify infinite loops, improves data-processing reliability, and reduces database migration failures.

10. Food Delivery App – State Transition Testing

Problem Analysis:

A mobile food delivery application uses a real-time tracker with states such as Ordered, Preparing, Out for Delivery, and Delivered. Users report that the tracker becomes stuck when a driver cancels during delivery. This indicates that the system may not handle all state transitions and exceptional events correctly.

1. State Transition Testing

State Transition Testing verifies how a system changes from one state to another in response to events. For a food delivery application, the main states include:

- Ordered: The customer places an order.
- Preparing: The restaurant prepares the food.
- Out for Delivery: The driver has collected the order.
- Delivered: The order has been delivered successfully.
- Cancelled or Reassignment Pending: The order cannot continue with the current driver.

2. State Transition Table

Current state	Event	Expected next state
Ordered	Restaurant accepts	Preparing
Preparing	Food is dispatched	Out for Delivery
Out for Delivery	Successful delivery	Delivered
Out for Delivery	Driver cancels	Reassignment pending or cancellation handling
Preparing	Permitted cancellation	Cancelled
Delivered	Repeated cancellation request	Remain Delivered

The exact transitions should follow the application's business rules.

3. Improved Testing Strategy

Test all valid transitions and important invalid transitions. Simulate driver cancellation, reassignment, repeated events, network interruptions, and delayed status updates. Verify that the application displays the correct state and does not remain stuck in an intermediate state.

The testing team should also check that the customer receives accurate notifications and that the order status remains consistent between the mobile application and backend server.

4. Justification

Equivalence Class Partitioning and Boundary Value Analysis mainly examine input categories and numerical limits. They may not adequately cover sequences of events. State Transition Testing specifically examines how previous states and events influence the next state.

Conclusion:

State Transition Testing helps detect missing transitions, invalid state changes, and workflow failures. It improves real-time tracking, handles driver cancellations more reliably, and provides a better experience for customers.

11. Cybersecurity Authentication Module – Hybrid Testing Strategy

Problem Analysis:

A cybersecurity authentication module uses password strength, OTP timing, and IP location to verify users. Despite testing, brute-force vulnerabilities were discovered. This indicates that the existing testing strategy may not adequately cover repeated login attempts, authentication bypasses, and different execution paths. A hybrid testing approach combining black-box and white-box techniques is required.

1. Black-Box Testing

Black-box testing verifies the system's behavior without examining its internal code. The following cases should be tested:

- Valid and invalid passwords.
- Correct, incorrect, and expired OTPs.
- Login attempts from unfamiliar IP addresses.
- Repeated failed login attempts.
- Attempts to reuse an OTP or bypass authentication steps.

These tests help identify weaknesses in the authentication process.

2. Error Guessing

Error Guessing uses the tester's experience to identify likely defects. The team should test repeated password attempts, missing OTPs, expired tokens, and attempts to access protected pages without authentication.

It should also verify whether account lockout, rate limiting, and appropriate security alerts are implemented.

3. White-Box Testing and Path Analysis

Construct a Control Flow Graph (CFG) representing password validation, OTP verification, IP checking, and access authorization. Identify independent paths and test both successful and unsuccessful authentication routes.

The system must grant access only when all required authentication conditions are satisfied. Branch coverage should be measured to identify untested decision outcomes.

4. Improved Testing Strategy

Combine error guessing with path analysis and automated security tests. Test authentication failures, repeated requests, session expiration, and recovery procedures. Verify that sensitive information is not exposed through error messages and that security controls cannot be bypassed.

Conclusion:

Combining black-box testing, error guessing, and white-box path analysis improves the detection of authentication weaknesses. This approach strengthens access control, reduces brute-force risks, and improves the overall security of the application.

12. E-Learning Platform – Equivalence Class Partitioning

Problem Analysis:

An e-learning platform provides Free, Basic, and Premium access based on subscription dates and course types. Errors occur when users upgrade exactly on the expiration date. This indicates inadequate testing of date boundaries and subscription transitions.

1. Equivalence Class Partitioning (ECP)

ECP divides inputs into groups expected to behave similarly. The subscription dates can be divided into the following classes:

- Before expiration: The subscription is active.
- On the expiration date: Access and upgrade eligibility depend on the defined expiration policy.
- After expiration: The subscription is expired unless renewed.
- Invalid dates: Missing or incorrectly formatted dates must be rejected.

Subscription types should also be divided into Free, Basic, and Premium classes. Each class must be tested against the permitted course-access rules.

2. Test Case Design

Test scenario	**Expected result**
Upgrade before expiration	Process according to upgrade rules
Upgrade on expiration date	Apply the defined date policy
Upgrade after expiration	Require renewal if necessary
Free user accesses Premium course	Deny restricted access
Valid Basic-to-Premium upgrade	Grant Premium access after successful processing
Invalid subscription date	Reject or report the error

3. Improved Testing Strategy

Test all subscription types using dates before, on, and after expiration. Include renewals, upgrades, payment failures, and course-access checks. Verify that the subscription status, payment record, and access permissions remain consistent.

Boundary testing should also cover month-end dates, leap years, and time-zone differences where relevant.

4. Justification

Testing only ordinary subscription dates may miss defects at expiration boundaries. ECP provides systematic coverage of subscription categories, while boundary-focused tests examine critical date transitions.

Conclusion:

Combining equivalence partitioning with date-boundary testing improves subscription management. It prevents unexpected access failures, ensures correct upgrade processing, and provides a consistent learning experience.

13. Smart-Home Lighting Controller – Branch and Condition Coverage

Problem Analysis:

A smart-home lighting controller adjusts brightness using ambient light sensors, switch-case statements, and complex Boolean expressions. Relying only on statement coverage may leave some decision outcomes and logical combinations untested. This can cause incorrect brightness settings or failures when sensor readings are abnormal.

1. Limitations of Statement Coverage

Statement coverage verifies that every executable statement runs at least once. However, a switch-case statement may contain several branches, and a Boolean expression may behave differently under different input combinations.

Therefore, executing every statement does not guarantee that all possible decision outcomes have been tested.

2. Branch Coverage

Branch coverage ensures that every decision outcome is executed. For example, the controller should be tested when ambient light is low, moderate, and high. Every relevant switch-case branch should also be executed.

3. Condition Coverage

Condition coverage verifies that each individual Boolean condition evaluates to both true and false. For example, consider a lighting decision based on low ambient light and occupancy:

Light ON = LowAmbientLight AND OccupancyDetected

Low ambient light	Occupancy detected	Expected result
True	True	Light ON
True	False	Light OFF
False	True	Light OFF
False	False	Light OFF

These test cases verify the behavior of both conditions and their combinations.

4. Improved Testing Strategy

Construct a CFG for the lighting algorithm and identify all switch-case branches and Boolean decisions. Test different sensor readings, occupancy states, invalid sensor values, and combinations of conditions. Verify that the selected brightness is correct and that the controller handles sensor failures safely.

Hardware integration tests should confirm that software commands correctly control the actual lighting devices.

Conclusion:

Combining statement, branch, and condition coverage provides more complete logical testing. It helps identify missing decision outcomes, improves sensor-error handling, and reduces hardware-software integration failures.

14. Logistics Management System – Robust Boundary Value Analysis

Problem Analysis:

A logistics management system calculates shipping costs based on package weight, destination zone, and delivery speed. Testers failed to detect errors when weights were zero or negative. This indicates that the existing input validation tests did not adequately cover invalid boundary values.

1. Robust Boundary Value Analysis

Robust BVA tests values at and around the valid boundaries, including values outside the permitted range.

Assume the minimum valid weight is greater than zero and the maximum permitted weight is $W_{\max}$

Test input	Expected result
Negative weight	Reject
Zero weight	Reject
Smallest valid positive weight	Accept
Value just above the minimum	Accept
Maximum permitted weight	Accept
Value just above the maximum	Reject

The exact limits should be determined by the logistics system's requirements.

2. Importance of Negative Testing

Negative testing verifies how the system behaves when invalid inputs are provided. For example, a negative package weight must not produce a shipping charge or be accepted as a valid shipment. Other invalid cases include missing weight, non-numeric values, and unsupported delivery options.

3. Improved Testing Strategy

Apply robust BVA to package weight and test the calculations across different destination zones and delivery speeds. Verify that valid inputs produce the correct shipping cost and invalid inputs generate appropriate error messages.

Automated tests should confirm that invalid data is rejected before the shipping calculation is performed.

4. Justification

Testing only normal positive weights may overlook validation defects. Robust BVA deliberately examines invalid values near the boundaries, helping detect errors that ordinary functional tests might miss.

Conclusion:

Robust Boundary Value Analysis improves input validation and prevents incorrect shipping calculations. It increases the reliability of the logistics system and ensures that invalid shipment details are handled safely.

15. Medical Imaging Software – Basis Path Testing

Problem Analysis:

Medical imaging software processes large datasets using parallel processing loops. The system produces inconsistent results when the dataset size is a prime number. This suggests possible defects in data partitioning, loop conditions, synchronization, or the merging of parallel results. Random testing alone may fail to reproduce these specific conditions consistently.

1. Limitations of Random Testing

Random testing selects inputs without following a fixed testing structure. Although it can reveal unexpected defects, it may not systematically cover independent execution paths or specific dataset sizes.

Therefore, structured white-box testing is necessary to examine the internal processing logic.

2. Basis Path Testing

Basis Path Testing identifies a set of independent execution paths through a program. The testing team should first construct a Control Flow Graph (CFG) representing data validation, partitioning, parallel processing, and result merging.

Cyclomatic complexity can be calculated using:

$$V(G) = E - N + 2P$$

Where:

- (E) is the number of edges.

- (N) is the number of nodes.
- (P) is the number of connected components.

The result estimates the number of independent paths required for basis path testing.

3. Improved Testing Strategy

The team should test:

- Empty and very small datasets.
- Datasets with prime and non-prime sizes.
- Datasets containing different numbers of records.
- Loop boundaries and incomplete data partitions.
- Parallel execution and synchronization conditions.
- The merging of partial results into the final output.

The computed results should be compared with a trusted reference implementation to verify mathematical accuracy. Repeated runs should also be checked for inconsistent results.

4. Justification

Basis Path Testing systematically covers independent control-flow paths, while targeted dataset tests investigate the prime-size condition. Together, these methods provide better coverage than random testing alone. However, additional concurrency testing may be required to detect synchronization defects.

Conclusion:

Combining Basis Path Testing, CFG analysis, dataset-boundary tests, and result validation helps identify parallel-processing defects. This approach improves mathematical accuracy, processing stability, and the reliability of medical imaging software.